

Research Program Repository Plan and Specifications

*Full Program Blueprint — Milestones, Budget, Architecture,
Staffing, and Risk*

Author: Robert Quinn, MBA

Date: May 2026

Version: 1.0

Classification: Internal — Research Engineering

Status: Approved for Distribution

Table of Contents

1	Executive Summary	\$1
2	Prioritized Milestones	\$2
3	Compute Budget	\$3
4	Repository Structure	\$4
5	Code Skeletons	\$5
6	CI/CD Design	\$6
7	Reproducible Notebooks	\$7
8	Test Suite	\$8

9	Staffing Plan	§9
10	Risk Register	§10
11	Next-Step Checklist	§11
A	Appendix A — Glossary	App. A
B	Appendix B — Tool and Library Versions	App. B
C	Appendix C — Useful Commands Reference	App. C

1. Executive Summary

This document establishes the authoritative plan and technical specifications for the Research Program Repository — a structured, reproducible, and engineering-grade codebase designed to support the full lifecycle of a machine learning research program. The program encompasses six months of targeted development, spanning data ingestion through peer-reviewed publication, with a principal emphasis on reproducibility, scientific rigor, and engineering best practices. All artifacts produced — trained model checkpoints, evaluation results, executed notebooks, and published reports — must be traceable to a specific version of code, configuration, and dataset, satisfying the standards expected by major ML venues including NeurIPS, ICML, and ICLR.

The end-to-end research pipeline proceeds through four primary stages: **(1) Data Ingestion**, in which raw datasets are acquired, validated against schema and quality contracts, and transformed into reproducible processed splits; **(2) Model Training**, in which baseline and experimental model architectures are implemented, configured via Hydra, and trained on provisioned GPU clusters with all hyperparameters, metrics, and artifacts logged to an experiment tracking platform (MLflow or Weights & Biases); **(3) Evaluation**, in which a standardized evaluation harness runs model checkpoints against a curated benchmark suite to produce

metric tables, error analyses, and leaderboard-ready results; and **(4) Publication and Deployment**, in which executed reproducible notebooks, model cards, dataset cards, and a manuscript are assembled and submitted to the target venue, with final checkpoints archived to long-term storage. Each stage is gated by automated CI/CD quality checks, ensuring that no unreviewed or untested code enters the main branch.

The program is staffed by a cross-functional team of 3.5 FTE research and engineering personnel, supported by 0.5 FTE in data engineering and technical writing, operating under the scientific direction of a 0.25 FTE Principal Investigator. The total compute budget is estimated at approximately **\$38,400 over six months**, subject to spend gate policies defined in Section 3. Key risks — including GPU quota shortfalls, data licensing constraints, and publication timeline slippage — are catalogued in the Risk Register (Section 10) with assigned owners and mitigation strategies. The program's success criteria are clear: a reproducible, fully documented codebase; a published or submitted research paper; and a model card-equipped artifact set that meets community standards for transparency and reusability.

Program At a Glance

Program Duration

6 Months (Month 1 – Month 6)

Primary Domain

Machine Learning Research & Engineering

Total Compute Budget

~\$38,400 (6-month estimate)

Team Size

6 roles / ~3.5 FTE

Target Output

Reproducible codebase, benchmarked model, peer-reviewed paper

Primary Tracking

MLflow / Weights & Biases

CI/CD Platform

GitHub Actions

Min. Test Coverage

80% line coverage (enforced in CI)

2. Prioritized Milestones

The following milestone table defines the full program execution schedule. Priority designations follow a P0/P1/P2 convention: **P0** milestones are critical-path blockers with zero schedule tolerance; **P1** milestones are high-value but can absorb up to one week of slip before impacting downstream work; **P2** milestones are important but non-blocking. All milestones are owned by a named role; the owner is accountable for delivery and for raising risks at least five business days before the target date.

ID	Milestone Name	Description	Priority	Target Date	Owner	Status
M01	Repository Scaffolding & CI/CD Setup	Initialize GitHub repo with branch protection, pyproject.toml,	P0	Month 1, Wk 1	Eng. Lead	<i>Planned</i>

ID	Milestone Name	Description	Priority	Target Date	Owner	Status
		Makefile, pre-commit hooks, and baseline GitHub Actions workflows (ci.yml, lint.yml).				
M02	Data Pipeline Ingestion & Validation	Implement DataIngestionPipeline (ingestion.py, preprocessing.py, validation.py), run against target dataset, validate schema contracts and pass quality thresholds.	P0	Month 1-2	Data Engineer	<i>Planned</i>
M03	Baseline Model Implementation	Implement BaseModel and first concrete architecture (e.g., transformer.py). Model must pass forward-pass smoke	P0	Month 2	ML Scientist	<i>Planned</i>

ID	Milestone Name	Description	Priority	Target Date	Owner	Status
		test and be registrable via registry.py.				
M04	Experiment Tracking Integration	Integrate MLflow or W&B SDK into Trainer. All runs must log hyperparameters, training/validation loss curves, and artifact paths. Workspace and project created.	P1	Month 2	MLOps / RA	Planned
M05	First Reproducible Training Run	Complete end-to-end training run from raw data to logged checkpoint with all metrics, seeds, and config recorded. Run must be reproducible by any team member from a single	P0	Month 2-3	ML Scientist	Planned

ID	Milestone Name	Description	Priority	Target Date	Owner	Status
		command.				
M06	Hyperparameter Sweep Infrastructure	Implement sweep configuration (configs/sweep/) using Hydra multirun or W&B sweeps. Execute first automated sweep on baseline model; results logged and visualized.	P1	Month 3	MLOps / RA	Planned
M07	Evaluation Harness & Benchmark Suite	Implement Evaluation Harness with full benchmark suite (benchmarks.py, metrics.py). Validate against at least two published baselines; generate reproducible leaderboard table.	P0	Month 3-4	ML Scientist	Planned

ID	Milestone Name	Description	Priority	Target Date	Owner	Status
M08	Reproducible Notebooks Publication	All five planned notebooks (NB01–NB05) executed via Papermill, cleared of secrets, and published to GitHub Pages or S3. Notebook execution integrated into CI (publish-notebooks.yml).	P1	Month 4	Eng. Lead	<i>Planned</i>
M09	Model Card & Dataset Card Authoring	Author docs/model_card.md and docs/dataset_card.md per Hugging Face community standards. Cards must include intended use, limitations, evaluation results, and licensing.	P1	Month 4–5	Tech. Writer	<i>Planned</i>
M10	Peer Review &	Internal paper	P2	Month 6	PI	<i>Planned</i>

ID	Milestone Name	Description	Priority	Target Date	Owner	Status
	External Publication Submission	draft reviewed by PI and Eng. Lead. Manuscript formatted to target venue template (LaTeX). Submission package includes code, model card, and reproducibility checklist.				

Priority Legend

P0 — Critical path, zero-tolerance for slip. | **P1** — High-value, up to 1-week slip tolerated. | **P2** — Important, non-blocking upstream milestones.

3. Compute Budget

The compute budget below reflects estimated costs for a six-month research program conducted primarily on a major cloud provider (AWS, GCP, or Azure). All estimates assume on-demand pricing for reserved instances and discounted spot/preemptible pricing for sweep workloads. Unit costs reflect 2025-2026 cloud

pricing benchmarks and should be validated against the team's negotiated enterprise rates before program kickoff. Costs are denominated in USD.

Resource	Unit	Qty	Unit Cost (USD)	Duration	Total Cost (USD)
GPU Cluster — A100 80GB × 8 nodes (training runs, on-demand reserved)	Node-hour	1	\$32.77 / hr	~120 hrs/mo	\$3,932 / mo
Preemptible / Spot GPU Instances (hyperparameter sweeps, 70% discount)	Node-hour (spot)	4	\$9.83 / hr	~80 hrs/mo	\$786 / mo
CPU Instances — Data Preprocessing (c5.4xlarge or equiv.)	Instance-hour	2	\$0.68 / hr	~200 hrs/mo	\$272 / mo
Object Storage — Datasets & Checkpoints (S3/GCS standard tier)	TB / month	5	\$23.00 / TB	6 months	\$115 / mo
NFS / Shared File Storage — Active training workspace (EFS or equiv.)	GB / month	500	\$0.30 / GB	6 months	\$150 / mo
Networking Egress — Data	TB egress	2	\$90.00 / TB	Monthly avg.	\$180 / mo

Resource	Unit	Qty	Unit Cost (USD)	Duration	Total Cost (USD)
transfers, checkpoint downloads					
MLOps Platform — Experiment Tracking SaaS (W&B Teams or MLflow Cloud)	Seat / month	6	\$50.00 / seat	6 months	\$300 / mo
Cloud Monitoring & Logging (CloudWatch, Stackdriver, or equiv.)	Flat / month	1	\$65.00 / mo	6 months	\$65 / mo
Total Monthly Estimate (Blended Average)	\$5,800 / mo				
<i>Note: Month 1 lower (~\$2,100, setup & pipeline only). Months 2-5 peak (~\$6,400-\$7,200). Month 6 lower (~\$3,200, writeup).</i>	<i>Variable</i>				
Total 6-Month Program Estimate	~\$38,400				

3.1 Budget Policy

All compute expenditure is governed by the following spend gate and approval policy, effective from program Day 1:

- **50% Utilization Gate (\$19,200):** When cumulative spend reaches 50% of the approved 6-month budget, the Research Engineer Lead must generate a spend vs. forecast variance report and present it to the PI within two business days. If variance exceeds 15% above forecast, a corrective action plan is required before further provisioning.
- **80% Utilization Gate (\$30,720):** When cumulative spend reaches 80% of the approved budget, all new compute provisioning requests above \$500 require explicit written approval from the PI and a designated budget authority (e.g., department chair, grants administrator). A revised budget forecast covering the remaining program period must be submitted.
- **Single-Purchase Approval Threshold:** Any individual cloud resource provisioning exceeding **\$2,500** in a single request requires prior written approval from the PI regardless of cumulative utilization level.
- **Reserved vs. Spot Ratio:** Maintain a target ratio of approximately **60% reserved / 40% spot** GPU hours. Reserved capacity covers primary training runs where interruption is unacceptable; spot capacity covers hyperparameter sweeps and exploratory runs that are checkpointed every 15 minutes and designed to tolerate interruption.
- **Cost Tagging:** All cloud resources must be tagged with project, milestone, and owner labels at provisioning time to enable per-milestone cost attribution in the monitoring dashboard.

Cost Overrun Risk

Hyperparameter sweeps are the highest cost-volatility activity. Sweeps must be configured with explicit **maximum wall-clock time** and **maximum concurrent workers** limits before launch. Unbounded sweep configurations

must not be submitted to production compute. See Risk R06 in Section 10.

4. Repository Structure

The following canonical directory layout governs all code, configuration, data references, and documentation artifacts in the research repository. All contributors are required to place new files within the appropriate directory and must not create top-level directories without an Engineering Lead review. The structure is designed to be immediately legible to any researcher or engineer unfamiliar with the project.

```
research-repo/ |— .github/ | | | |— workflows/ | | | |— ci.yml
# Push/PR: lint, type-check, unit & integration tests | | | |— lint.yml
# Pre-commit, ruff, black, isort checks | | | |— publish-notebooks.yml #
On merge to main: execute & publish notebooks | | | |— CODEOWNERS
# Auto-assign reviewers by directory |— configs/ | | | |— base.yaml
# Shared base configuration (merged by Hydra) | | | |— model/
# Model-specific config overrides | | | |— sweep/ # Hydra
multirun sweep configurations |— data/ | | | |— raw/ #
Immutable raw data (never modified, gitignored) | | | |— processed/
# Reproducibly generated processed splits | | | |— external/
# Third-party reference datasets |— notebooks/ | | | |—
01_data_exploration.ipynb | | | |— 02_baseline_training.ipynb | | | |—
03_evaluation_analysis.ipynb | | | |— 04_ablation_study.ipynb |— src/ | | | |—
data/ | | | |— __init__.py | | | |— ingestion.py #
DataIngestionPipeline | | | |— preprocessing.py # Feature
transforms, tokenization | | | |— validation.py # Schema &
quality validation | | | |— models/ | | | |— __init__.py | | | |—
base_model.py # Abstract BaseModel interface | | | |—
transformer.py # Concrete transformer architecture | | | |—
registry.py # Model registry (name → class) | | | |— training/ |
| | | |— __init__.py | | | |— trainer.py # Training loop
orchestrator | | | |— optimizer.py # Optimizer factory | | | |—
|— scheduler.py # LR scheduler factory | | | |— evaluation/ | | | |—
|— __init__.py | | | |— metrics.py # Metric computation
functions | | | |— harness.py # EvaluationHarness
orchestrator | | | |— benchmarks.py # Benchmark dataset loaders
| | | |— utils/ | | | |— __init__.py | | | |— logging.py #
Structured logging setup | | | |— seed.py # Global RNG
seed utilities | | | |— config.py # Hydra/OmegaConf config
dataclasses |— tests/ | | | |— unit/ # Pure function
tests (~70% of suite) | | | |— integration/ # Pipeline end-to-
end with fixtures (~20%) | | | |— smoke/ # Full loop on
tiny synthetic data (~10%) |— scripts/ | | | |— train.sh #
Launch training run (wraps python -m src.training) | | | |— evaluate.sh
```

```

# Run evaluation harness on a checkpoint | └─ sweep.sh
# Launch Hydra multirun sweep └─ docs/ | └─ model_card.md #
Model card (Hugging Face format) | └─ dataset_card.md #
Dataset card with provenance & license └─ pyproject.toml #
Project metadata, deps, tool configuration └─ Makefile
# Developer workflow shortcuts └─ README.md # Project
overview, quickstart, citations └─ .pre-commit-config.yaml # Pre-
commit hook definitions

```

4.1 Top-Level Directory Descriptions

Directory / File	Purpose and Conventions
.github/	All GitHub-specific automation. Workflows enforce quality gates on every PR. CODEOWNERS assigns mandatory reviewers: the Eng. Lead owns src/ and .github/; the ML Scientist owns src/models/ and configs/.
configs/	Hydra configuration hierarchy. base.yaml provides defaults; model- and sweep-specific overrides compose on top. Never hard-code hyperparameters in source code — all values live here.
data/	Data pointers and lightweight metadata only — raw data files are gitignored and managed via DVC or an S3 manifest. raw/ is immutable; all transformations are idempotent and rerunnable.
notebooks/	Numbered Jupyter notebooks for exploration, training demos, evaluation, and ablations. Notebooks are Papermill-parameterized and committed in executed form. Secrets are never present; outputs are reproducible.
src/	All importable Python source code. Organized into four domain packages (data, models, training, evaluation) and a utils package. No notebook-specific logic in src/.
tests/	Full test suite organized by pyramid level. Mirrors the src/ structure within unit/ and integration/. Smoke tests in smoke/ run against tiny synthetic data in under 60 seconds.
scripts/	Thin shell wrappers for common

Directory / File	Purpose and Conventions
	operations. Scripts set environment variables, activate the correct Conda environment, and delegate to Python modules. Not for complex logic.
docs/	Human-readable documentation artifacts. The model card and dataset card are versioned alongside code so that changes to the model architecture or data pipeline are reflected in documentation atomically.
pyproject.toml	Single source of truth for package metadata, dependencies, and tool configuration (pytest, ruff, mypy, black, isort). Replaces setup.py, setup.cfg, and multiple config files.
Makefile	Developer workflow shortcuts. See Appendix C for the full target reference. Targets include make train, make test, make lint, make notebook, and make sweep.

5. Code Skeletons

The following Python module skeletons define the public interface for each core component. All implementations must conform to these interfaces. Method signatures, type annotations, and docstring conventions are normative; implementation bodies are marked with `# TODO:` comments indicating required behavior. All modules require Python 3.10+ and must be fully compatible with the pinned dependency versions listed in Appendix B.

5.1 src/data/ingestion.py

```
""" src/data/ingestion.py ===== Data ingestion pipeline for
the research program repository. Responsibilities:
- Load raw data from configured sources (local, S3, HuggingFace Hub).
- Validate loaded data against schema and quality contracts.
- Produce reproducible train/validation/test splits.
- Persist processed splits to the
```

```

configured output directory. All operations must be idempotent: re-running
on the same input produces bit-identical output (given fixed random seeds).
""" from __future__ import annotations import logging from pathlib import
Path from typing import Any import pandas as pd from omegaconf import
DictConfig logger = logging.getLogger(__name__) class
DataIngestionPipeline: """Orchestrates the full data ingestion workflow.
Args: cfg: Hydra DictConfig containing data configuration block.
Expected keys: raw_dir, processed_dir, split_ratios, seed. """ def
__init__(self, cfg: DictConfig) -> None: self.cfg = cfg
self.raw_dir = Path(cfg.data.raw_dir) self.processed_dir =
Path(cfg.data.processed_dir) self._data: pd.DataFrame | None = None
# TODO: Initialize any dataset-specific metadata (schema, expected columns).
def load_raw(self) -> pd.DataFrame: """Load raw data from the
configured source path or remote URI. Returns:
pd.DataFrame: Unprocessed raw dataframe with original column names.
Raises: FileNotFoundError: If raw_dir does not exist or contains
no files. ValueError: If loaded data is empty. # TODO:
Support loading from local path, S3 URI, and HuggingFace Hub dataset name.
# TODO: Log row count, column names, and dtypes at INFO level after loading.
# TODO: Store loaded data in self._data for downstream steps. """
pass def validate(self) -> None: """Run schema and data quality
validation on the loaded raw data. Checks performed: -
Required columns present with expected dtypes. - No null values
in required fields beyond configured threshold. - Value ranges
within expected bounds. - No duplicate primary keys.
Raises: ValueError: If any validation check fails. Error message
must enumerate all failing checks (not just the
first). # TODO: Implement schema validation using pandera or Great
Expectations. # TODO: Log validation report as structured JSON at
DEBUG level. # TODO: Emit a summary pass/fail metric to experiment
tracker. """ pass def split(self) -> dict[str,
pd.DataFrame]: """Produce reproducible train/validation/test splits.
Returns: dict[str, pd.DataFrame]: Keys are 'train', 'val',
'test'. Ratios sourced from
cfg.data.split_ratios. # TODO: Use cfg.data.seed for the random
split to ensure reproducibility. # TODO: Stratify by label column if
cfg.data.stratify is True. # TODO: Assert that split sizes sum to
total row count. """ pass def save_processed(self,
splits: dict[str, pd.DataFrame]) -> None: """Persist processed splits
to the configured output directory. Args: splits:
Dictionary of split name -> DataFrame, as returned by split(). #
TODO: Save each split as Parquet (preferred) or CSV at self.processed_dir /
{split}.parquet. # TODO: Write a manifest JSON file recording split
sizes, column schemas, and git SHA. # TODO: Log output paths and file
sizes at INFO level. """ pass

```

5.2 src/models/base_model.py

```

""" src/models/base_model.py ===== Abstract base class
defining the model interface for all architectures implemented in this
research program. All concrete model classes must subclass BaseModel and
implement every abstract method. This interface is the contract consumed by
Trainer and EvaluationHarness – do not change method signatures without a
breaking- change review. """ from __future__ import annotations from abc
import ABC, abstractmethod from typing import Any import torch import
torch.nn as nn from omegaconf import DictConfig class BaseModel(ABC,
nn.Module): """Abstract base class for all research models. Args:
cfg: Hydra DictConfig containing the model configuration block.
Expected keys vary by architecture; see concrete implementations. """

```



```

def __init__(self, cfg: DictConfig) -> None:
    self.cfg = cfg
    # TODO: Parse and store common model hyperparameters
    (hidden_dim, num_layers, dropout). @abstractmethod def forward(self,
    batch: dict[str, torch.Tensor]) -> dict[str, torch.Tensor]:
    """Perform a forward pass. Args: batch: Dictionary of
    string keys to tensors. At minimum must contain
    'input_ids' or 'features'. May include 'attention_mask'. Returns:
    Dictionary of output tensors. Must contain key 'logits'. # TODO:
    Implement in each concrete subclass. # TODO: Ensure output dict keys
    are documented in the model card. """ pass
    @abstractmethod def compute_loss(
    self, outputs:
    dict[str, torch.Tensor], batch: dict[str, torch.Tensor], ) ->
    torch.Tensor: """Compute the training loss from model outputs and
    ground truth labels. Args: outputs: Output dict from
    forward(), must contain 'logits'. batch: Input batch dict, must
    contain 'labels'. Returns: Scalar loss tensor
    (requires_grad=True). # TODO: Support task-specific loss functions
    (cross-entropy, MSE, contrastive). # TODO: Apply label smoothing if
    cfg.model.label_smoothing > 0. """ pass @abstractmethod
    def configure_optimizers(self) -> dict[str, Any]: """Create and
    return optimizer (and optionally scheduler) for training. Returns:
    Dictionary with keys: 'optimizer': torch.optim.Optimizer
    instance. 'lr_scheduler' (optional): LR scheduler dict
    compatible with PyTorch Lightning
    or manual step(). # TODO: Support optimizer choice from
    cfg.training.optimizer (adamw, sgd, lion). # TODO: Apply weight decay
    only to non-bias, non-LayerNorm parameters. # TODO: Implement
    parameter group differentiation (e.g., lower LR for embeddings). """
    pass def count_parameters(self) -> int: """Return total number
    of trainable parameters.""" return sum(p.numel() for p in
    self.parameters() if p.requires_grad)

```

5.3 src/training/trainer.py

```

""" src/training/trainer.py ===== Training loop
orchestrator for the research program. The Trainer manages the full training
lifecycle: epoch iteration, gradient updates, validation, checkpointing, and
experiment tracking. It is designed to be framework-agnostic (pure PyTorch),
with optional integration points for Accelerate or Lightning via composition.
Design invariants:
- All random state is seeded before the first step
(via utils.seed).
- Every run logs to the configured experiment tracker
(MLflow / W&B).
- Checkpoints are saved atomically (write to temp path,
then rename). """
from __future__ import annotations
import logging
from pathlib import Path
from typing import Any
import torch
from omegaconf import DictConfig
from torch.utils.data import DataLoader
from src.models.base_model import BaseModel
from src.utils.seed import set_global_seed
logger = logging.getLogger(__name__)

class Trainer:
    """Manages the training and validation loop. Args: model:
    Instance of a BaseModel subclass. cfg: Hydra DictConfig with
    training configuration block. train_loader: DataLoader for training
    split. val_loader: DataLoader for validation split. """
    def __init__(
        self, model: BaseModel, cfg: DictConfig,
        train_loader: DataLoader, val_loader: DataLoader, ) -> None:
        self.model = model
        self.cfg = cfg
        self.train_loader = train_loader
        self.val_loader = val_loader
        self.device = torch.device(cfg.training.device)
        self.checkpoint_dir = Path(cfg.training.checkpoint_dir)
        # TODO: Initialize optimizer and scheduler from model.configure_optimizers().
        # TODO: Initialize experiment tracker (MLflow run or W&B run) with cfg hyperparams.

```

```

TODO: Move model to self.device.          # TODO: Call
set_global_seed(cfg.training.seed) to fix RNG state.          def
train_epoch(self, epoch: int) -> dict[str, float]:          """Run one full
training epoch over train_loader.          Args:          epoch: Current
epoch index (0-based).          Returns:          dict of metric name ->
float value for this epoch (e.g., {'train_loss': 0.42}).          # TODO: Set
model.train() mode.          # TODO: Accumulate loss over batches; handle
gradient accumulation steps.          # TODO: Clip gradients if
cfg.training.grad_clip_norm is set.          # TODO: Log per-step loss to
experiment tracker every cfg.training.log_every_n_steps.          """
pass          def validate_epoch(self, epoch: int) -> dict[str, float]:
"""Run one full validation pass over val_loader.          Args:
epoch: Current epoch index (0-based).          Returns:          dict of
metric name -> float value (e.g., {'val_loss': 0.38, 'val_acc': 0.91}).
# TODO: Set model.eval() mode and wrap in torch.no_grad().          # TODO:
Compute all metrics configured in cfg.evaluation.metrics.          # TODO: Log
validation metrics to experiment tracker.          """          pass          def
fit(self) -> dict[str, Any]:          """Run the full training loop for
cfg.training.max_epochs epochs.          Returns:          dict containing
final training and validation metrics, best          checkpoint path, and
total wall-clock training time.          # TODO: Implement early stopping if
cfg.training.early_stopping is enabled.          # TODO: Save best checkpoint
based on cfg.training.monitor_metric.          # TODO: Log total training time
and final metrics to experiment tracker.          # TODO: Call
self.save_checkpoint() at the end of each epoch.          """          pass
def save_checkpoint(self, epoch: int, metrics: dict[str, float]) -> Path:
"""Save model state, optimizer state, and metadata atomically.          Args:
epoch:          Epoch index at time of save.          metrics: Metric dict to
embed in checkpoint metadata.          Returns:          Path to the saved
checkpoint file.          # TODO: Write to a temp file first, then atomically
rename.          # TODO: Include: model state_dict, optimizer state_dict,
epoch, cfg, metrics, git SHA.          # TODO: Prune old checkpoints to keep
only cfg.training.keep_last_n checkpoints.          """          pass          def
load_checkpoint(self, checkpoint_path: Path) -> int:          """Restore model
and optimizer state from a checkpoint file.          Args:
checkpoint_path: Path to the checkpoint file (*.pt or *.ckpt).
Returns:          int: Epoch number to resume from (i.e., saved epoch +
1).          # TODO: Load state dicts for model and optimizer.          #
TODO: Validate that checkpoint cfg matches current cfg (warn on mismatch).
# TODO: Log checkpoint path and embedded metrics at INFO level.          """
pass

```

5.4 src/evaluation/harness.py

```

""" src/evaluation/harness.py ===== Evaluation harness
orchestrating benchmark execution, metric computation, and report generation.
The EvaluationHarness is the authoritative component for producing the
results tables that appear in the paper and model card. All benchmark results
must be generated through this harness to ensure reproducibility and
comparability across experimental runs. """ from __future__ import
annotations import logging from pathlib import Path from typing import Any
import pandas as pd import torch from omegaconf import DictConfig from
src.models.base_model import BaseModel from src.evaluation.benchmarks import
load_benchmark from src.evaluation.metrics import compute_metrics logger =
logging.getLogger(__name__) class EvaluationHarness:          """Orchestrates
evaluation across the full benchmark suite.          Args:          model:
Trained model checkpoint (BaseModel instance, eval mode).          cfg:
Hydra DictConfig with evaluation configuration block.          output_dir:
Directory to write evaluation reports and artifacts.          """          def

```

```

__init__(self, model: BaseModel, cfg: DictConfig,
output_dir: Path, ) -> None: self.model = model self.cfg
= cfg self.output_dir = output_dir self.results: dict[str,
dict[str, float]] = {} # TODO: Set model to eval mode and move to
configured device. # TODO: Validate that output_dir exists; create if
not. def run_benchmarks(self) -> dict[str, dict[str, float]]:
"""Execute all benchmarks listed in cfg.evaluation.benchmarks.
Returns: Nested dict: {benchmark_name: {metric_name:
float_value}}. # TODO: Iterate over cfg.evaluation.benchmarks list.
# TODO: For each benchmark, load via load_benchmark(name, cfg), run
inference, # compute metrics, and store in self.results.
# TODO: Log per-benchmark results to experiment tracker. # TODO:
Handle benchmark failures gracefully – log error and continue. """
pass def compute_metrics(self, predictions: list[Any],
references: list[Any], benchmark_name: str, ) -> dict[str,
float]: """Compute the configured metric set for a single benchmark.
Args: predictions: Model output predictions (decoded tokens or
logits). references: Ground truth labels or reference
strings. benchmark_name: Name used to select the correct metric
config. Returns: dict mapping metric name to scalar
float value. # TODO: Route to appropriate metric functions based on
benchmark config. # TODO: Support: accuracy, F1 (macro/micro), BLEU,
ROUGE, perplexity, MCC. """ pass def
generate_report(self) -> Path: """Compile all benchmark results into
a structured evaluation report. Returns: Path to the
generated report file (Markdown and/or JSON). # TODO: Format
self.results as a leaderboard table (benchmark × metric). # TODO:
Include model metadata: checkpoint path, training config, git SHA. #
TODO: Write report.md and results.json to self.output_dir. # TODO:
Log the report path to the experiment tracker as an artifact. """
pass

```

5.5 src/utils/config.py

```

""" src/utils/config.py ===== Structured configuration
dataclasses for the research program. All configuration is managed via Hydra
+ OmegaConf. These dataclasses serve as the schema layer: Hydra validates
loaded YAML configs against them at startup, ensuring type correctness and
required field presence. Usage: # In main entry point:
@hydra.main(version_base=None, config_path="../..configs",
config_name="base") def main(cfg: DictConfig) -> None: config =
OmegaConf.structured(ProgramConfig) cfg = OmegaConf.merge(config,
cfg) """ from __future__ import annotations from dataclasses import
dataclass, field from typing import Optional @dataclass class ModelConfig:
"""Configuration for model architecture and initialization.""" name: str
= "transformer" # Model registry key hidden_dim: int = 768
# Hidden dimension size num_layers: int = 12 # Number
of encoder/decoder layers num_heads: int = 12 # Number
of attention heads dropout: float = 0.1 # Dropout
probability label_smoothing: float = 0.0 # Label smoothing
epsilon (0 = disabled) pretrained_weights: Optional[str] = None #
HuggingFace model ID or local path # TODO: Add architecture-specific
fields in subclassed configs (e.g., vocab_size). @dataclass class
DataConfig: """Configuration for data loading, splitting, and
preprocessing.""" raw_dir: str = "data/raw" processed_dir: str =
"data/processed" dataset_name: str = "REQUIRED" # Must be
overridden in run config split_ratios: dict =
field( default_factory=lambda: {"train": 0.8, "val": 0.1, "test":
0.1} ) stratify: bool = True seed: int = 42 num_workers: int

```

```

= 4      batch_size: int = 32      max_seq_len: int = 512      # TODO: Add
dataset-specific preprocessing flags (lowercase, normalize, etc.).
@dataclass class TrainingConfig:      """Configuration for the training loop
and optimization."""      max_epochs: int = 50      learning_rate: float = 1e-4
weight_decay: float = 0.01      optimizer: str = "adamw"      #
Options: adamw, sgd, lion      grad_clip_norm: Optional[float] = 1.0
warmup_steps: int = 1000      device: str = "cuda"      checkpoint_dir: str =
"checkpoints"      keep_last_n: int = 3      seed: int = 42
log_every_n_steps: int = 50      monitor_metric: str = "val_loss"
early_stopping: bool = True      patience: int = 5      # TODO: Add mixed
precision flag (fp16/bf16) and gradient accumulation steps.
@dataclass
class EvaluationConfig:      """Configuration for the evaluation harness."""
benchmarks: list = field(default_factory=list) # List of benchmark names to
run      metrics: list = field(      default_factory=lambda: ["accuracy",
"f1_macro"]      )      output_dir: str = "evaluation_outputs"      batch_size:
int = 64      # TODO: Add per-benchmark metric overrides.
@dataclass class
ProgramConfig:      """Root configuration dataclass composing all sub-
configs."""      model: ModelConfig = field(default_factory=ModelConfig)
data: DataConfig = field(default_factory=DataConfig)      training:
TrainingConfig = field(default_factory=TrainingConfig)      evaluation:
EvaluationConfig = field(default_factory=EvaluationConfig)
experiment_name: str = "baseline_run"      tracker: str = "mlflow"
# Options: mlflow, wandb, none      project: str = "research-program"

```

6. CI/CD Design

The CI/CD pipeline enforces code quality, test coverage, and reproducibility standards automatically on every pull request and merge event. No code may be merged to the `main` branch without passing all required status checks. The pipeline is implemented entirely in GitHub Actions and is defined as code within the `.github/workflows/` directory.

6.1 Pipeline Overview

The following table describes the end-to-end CI/CD flow from pull request creation through artifact publication:

Step	Stage	Description	Trigger	Workflow
1	Pull Request Opened	Developer opens PR from feature branch to	on: pull_request	—

Step	Stage	Description	Trigger	Workflow
		main. Branch protection rules require at least 1 approving review and all status checks to pass.		
2	Lint & Format Check	Runs ruff (linting), black --check (formatting), isort --check (import order), and mypy (static type checking). Any violation fails the check immediately.	push / PR	lint.yml
3	Unit Tests	Executes pytest tests/unit/ with coverage instrumentation. Requires $\geq 80\%$ line coverage on src/. Coverage report uploaded to Codecov.	push / PR	ci.yml
4	Integration Tests	Executes pytest tests/integration/ using small fixture datasets committed to the repo. Tests the pipeline end-to-end on CPU (no GPU required in CI).	push / PR	ci.yml
5	Smoke Tests	Executes	push / PR	ci.yml

Step	Stage	Description	Trigger	Workflow
		pytest tests/smoke/ — full train and eval loop on a tiny synthetic dataset. Must complete in under 60 seconds on a 2-core GitHub Actions runner.		
6	PR Review & Merge	Eng. Lead or designated reviewer approves the PR. CODEOWNERS enforcement ensures domain-appropriate reviewers. Merge is squash-commit to keep history linear.	on: merge	—
7	Notebook Execution & Publish	On merge to main, all notebooks are re-executed via Papermill in dependency order. Executed .ipy nb files are converted to HTML via nbconvert and published to GitHub Pages or S3.	on: push to main	publish-notebooks.yml

6.2 GitHub Actions Workflow Skeletons

ci.yml — Continuous Integration

```
name: CI on: push: branches: [main, develop] pull_request:
branches: [main] env: PYTHON_VERSION: "3.11" jobs: lint: name: Lint
& Type Check runs-on: ubuntu-latest steps: - uses:
actions/checkout@v4 - uses: actions/setup-python@v5 with:
python-version: ${ env.PYTHON_VERSION }} cache: pip - name:
Install dev dependencies run: pip install -e ".[dev]" - name:
Run ruff (linting) run: ruff check src/ tests/ - name: Run
black (formatting check) run: black --check src/ tests/ - name:
Run mypy (type checking) run: mypy src/ --ignore-missing-imports --
strict test: name: Unit & Integration Tests runs-on: ubuntu-latest
needs: lint steps: - uses: actions/checkout@v4 - uses:
actions/setup-python@v5 with: python-version: $
{{ env.PYTHON_VERSION }} cache: pip - name: Install
dependencies run: pip install -e ".[dev]" - name: Run unit
tests with coverage run: | pytest tests/unit/
tests/integration/ \ --cov=src \ --cov-report=xml \
--cov-fail-under=80 \ -v - name: Upload coverage to Codecov
uses: codecov/codecov-action@v4 with: file: coverage.xml
fail_ci_if_error: true smoke: name: Smoke Tests (Full Pipeline)
runs-on: ubuntu-latest needs: test timeout-minutes: 5 steps:
- uses: actions/checkout@v4 - uses: actions/setup-python@v5
with: python-version: ${ env.PYTHON_VERSION }} cache:
pip - name: Install dependencies run: pip install -e ".[dev]"
- name: Run smoke tests run: pytest tests/smoke/ -v --timeout=60
```

lint.yml — Pre-commit and Format Enforcement

```
name: Lint on: push: branches: [main, develop] pull_request: jobs:
pre-commit: name: Pre-commit Hooks runs-on: ubuntu-latest steps:
- uses: actions/checkout@v4 - uses: actions/setup-python@v5
with: python-version: "3.11" - name: Install pre-commit
run: pip install pre-commit - name: Run pre-commit on all files
run: pre-commit run --all-files # Runs hooks defined in .pre-commit-
config.yaml: # - black (formatting) # - isort (import
sorting) # - ruff (linting, replaces flake8) # - check-
yaml, check-toml, end-of-file-fixer # - no-commit-to-branch
(prevents direct push to main)
```

publish-notebooks.yml — Notebook Execution and Publication

```
name: Publish Notebooks on: push: branches: [main]
workflow_dispatch: # Allow manual trigger jobs: execute-and-publish:
name: Execute Notebooks and Publish runs-on: ubuntu-latest
environment: production steps: - uses: actions/checkout@v4 -
uses: actions/setup-python@v5 with: python-version: "3.11"
cache: pip - name: Install dependencies run: pip install -e ".
[dev]" papermill nbconvert - name: Execute notebooks via Papermill
env: EXPERIMENT_TRACKER_API_KEY: ${ secrets.WANDB_API_KEY }}
run: | for nb in notebooks/0*.ipynb; do echo
"Executing: $nb" papermill "$nb" "notebooks/executed/$(basename
$nb)" \ --kernel python3 \ --log-output \
--request-save-on-cell-execute done - name: Convert to HTML
via nbconvert run: | jupyter nbconvert --to html
notebooks/executed/*.ipynb \ --output-dir docs/notebooks/ -
name: Publish to GitHub Pages uses: peaceiris/actions-gh-pages@v4
```

```

with:      github_token: ${{ secrets.GITHUB_TOKEN }}
publish_dir: ./docs/notebooks      destination_dir: notebooks      #
Alternatively: upload to S3      # - name: Upload to S3      # run: aws
s3 sync docs/notebooks/ s3://${{ secrets.NOTEBOOK_BUCKET }}/notebooks/
# env:      # AWS_ACCESS_KEY_ID: ${{ secrets.AWS_ACCESS_KEY_ID }}
# AWS_SECRET_ACCESS_KEY: ${{ secrets.AWS_SECRET_ACCESS_KEY }}

```

6.3 Quality Gates

Gate	Tool	Threshold	Fail Behavior
Code Coverage	pytest-cov + Codecov	≥ 80% line coverage on src/	CI fails; PR cannot be merged. Coverage report posted as PR comment.
Static Type Checking	mypy (strict mode)	Zero type errors	CI fails at lint stage. Error list annotated on PR diff via mypy-github-action.
Linting	ruff	Zero warnings (all rules enabled per pyproject.toml)	CI fails at lint stage. Specific rule violations reported in job log.
Code Formatting	black, isort	Zero formatting diffs	CI fails. Developer must run <code>make lint</code> locally before re-pushing.
Smoke Test Pass	pytest (smoke/)	100% of smoke tests pass within 60 seconds	CI fails at smoke stage. Prevents broken training loop from merging to main.
Notebook Execution	Papermill	Zero cell errors in any notebook	Publish job fails; previous published version remains live. Slack alert sent.
Dependency Security	pip-audit (or Dependabot)	Zero critical/high CVEs in dependencies	PR blocked with security advisory. Eng. Lead must approve explicit exception.

6.4 Secrets and Environment Management

All sensitive credentials are managed as GitHub Actions Secrets scoped to the repository or deployment environment. The following conventions apply to all team members and automated processes:

- **GitHub Actions Secrets** — Store API keys, cloud credentials, and experiment tracker tokens under Settings → Secrets and variables → Actions. Reference in workflows as `${{ secrets.SECRET_NAME }}`. Secrets are never echoed to logs.
 - **Required Secrets** — The following must be configured before the first CI run:
 - `WANDB_API_KEY` or `MLFLOW_TRACKING_URI` + `MLFLOW_TRACKING_TOKEN`
 - `AWS_ACCESS_KEY_ID` / `AWS_SECRET_ACCESS_KEY` (or OIDC-based role assumption — preferred)
 - `CODECOV_TOKEN` for coverage uploads
 - `NOTEBOOK_BUCKET` (S3 bucket name, if applicable)
 - **Local Development** — Use a `.env` file (gitignored, documented in `.env.example`) loaded via `python-dotenv` or `direnv`. Never hard-code credentials in source files or notebook cells.
 - **Environment Scoping** — The `publish-notebooks.yml` workflow uses the production GitHub environment, which requires an additional manual approval gate before publishing. Development environments use a separate set of secrets with read-only cloud access.
 - **Rotation Policy** — All API keys and cloud access keys must be rotated every 90 days. The Eng. Lead is responsible for updating GitHub Secrets after rotation. Key expiry dates must be tracked in the team's security log.
-
-

7. Reproducible Notebooks

All notebooks in the research repository are first-class scientific artifacts, not scratch pads. They must be fully executable from scratch in any environment with the pinned dependency stack installed, produce identical outputs given the same inputs and seeds, and be free of hard-coded paths, credentials, and unversioned state. The table below inventories all planned notebooks with their purpose, dependencies, and expected runtime.

ID	Filename	Purpose	Inputs	Outputs	Est. Runtime
NB01	01_data_exploration.ipynb	Exploratory data analysis (EDA): distribution analysis, data quality checks, missing value audit, label balance.	Processed splits from data/processed/	Summary statistics tables, class distribution plots, quality report Markdown.	~5 min (CPU)
NB02	02_baseline_training.ipynb	End-to-end training run demonstration using baseline model and default hyperparameters. Illustrates experiment tracking integration.	Processed splits, configs/baseline.yaml, W&B/MLflow workspace	Trained model checkpoint, loss/metric curves, W&B run link.	30–60 min (GPU required)
NB03	03_evaluation_analysis.ipynb	Comprehensive benchmark evaluation using Evaluation Harness. Produces	Model checkpoint from NB02, benchmark dataset loaders	Leaderboard table (Markdown), per-class error analysis, confusion matrix.	~15 min (CPU/GPU)

ID	Filename	Purpose	Inputs	Outputs	Est. Runtime
		leaderboard table and error analysis visualizations.			
NB04	04_ablation_study.ipynb	Systematic component ablation: disabling attention heads, varying depth, removing preprocessing steps. Quantifies contribution of each component.	Multiple checkpoints from sweep runs, evaluation fixtures	Ablation delta table, sensitivity heatmap, narrative findings section.	Varies (30 min - 4 hr GPU)
NB05	05_hyperparameter_sensitivity.ipynb	Analysis of hyperparameter sweep results: learning rate sensitivity, batch size effects, warmup schedule comparison.	W&B sweep results or MLflow experiment CSV export	Sensitivity plots (LR vs. loss, batch size vs. throughput), best-config table.	~10 min (CPU)

7.1 Reproducibility Checklist

Every notebook must satisfy all items on the following checklist before it is committed to the repository or published. The notebook author is responsible for verifying compliance; the Eng. Lead verifies during PR review.

- **Fixed Random Seeds:** All notebooks call the project's `set_global_seed(cfg.data.seed)` utility in the first executable cell. Seeds cover Python's `random`, `numpy`, and `torch` RNG states.
 - **Pinned Environment:** Notebooks execute within the project's Conda environment or virtual environment defined by `pyproject.toml` and the companion `conda-lock.yml` or `requirements-lock.txt`. The environment is documented in the notebook's first Markdown cell with the output of `pip list` or `conda list`.
 - **Papermill Parameterization:** All key variables (checkpoint path, dataset split, seed, output directory) are defined in a single tagged "parameters" cell at the top of the notebook. Papermill can override these parameters at execution time without manual cell editing.
 - **No Hard-Coded Paths or Secrets:** All paths are constructed from `cfg` or environment variables. No API keys, access tokens, or personal paths are present in committed notebooks.
 - **Executed Notebooks Committed or Archived:** Notebooks are committed to the repository in their executed state (with outputs) after successful Papermill execution in CI. Alternatively, executed `.ipynb` files are uploaded to a versioned S3 prefix with the associated git SHA.
 - **Cell Execution Order:** Notebooks must be fully runnable from top to bottom with "Restart Kernel and Run All Cells." No hidden state dependencies between cells are permitted.
 - **Output Cleared Before Source Commit:** Source notebook files (committed to version control) have outputs stripped using `nbstripout` (enforced via pre-commit hook). Executed versions are separate artifacts.
-
-

8. Test Suite

The test suite is designed around the standard testing pyramid, with the vast majority of tests being fast, isolated unit tests, supported by a smaller layer of integration tests and a minimal but critical smoke test layer. All tests are written using `pytest` with fixtures defined in `conftest.py` files at each pyramid level. Tests must not require GPU resources, live network access, or external API credentials to run in CI.

8.1 Test Pyramid

Level	% of Suite	Scope	Characteristics
Unit Tests	~70%	Pure functions and methods in <code>src/data</code> , <code>src/models</code> , <code>src/utils</code>	No I/O, no external deps. All dependencies mocked. Run in <5 seconds total. Each test is independent and deterministic.
Integration Tests	~20%	Pipeline components composed end-to-end using small fixture datasets (10-100 rows)	May write to a temp directory. Use real function calls but with minimal data. Run in <30 seconds total. Fixtures committed to <code>tests/fixtures/</code> .
Smoke Tests	~10%	Full train → evaluate loop on tiny synthetic dataset generated programmatically	Exercises all major code paths with real data flow. Must complete in <60 seconds on a 2-core CPU runner. Validates nothing is broken end-to-end.

8.2 Test Inventory

ID	Module	Test Name	Type	Description	Pass Criteria
T01	src/data/ingestion.py	test_load_raw_returns_dataframe	Unit	Calls load_raw() with a mock CSV source. Validates return type and non-empty result.	Returns non-empty pd.DataFrame; no exception raised.
T02	src/data/validation.py	test_validate_passes_on_clean_data	Unit	Passes a well-formed fixture DataFrame through validate(). Confirms no exception raised.	Completes without raising ValueError.
T03	src/data/validation.py	test_validate_raises_on_missing_column	Unit	Removes a required column from fixture DataFrame and passes to validate(). Checks for ValueError.	Raises ValueError with message identifying missing column.
T04	src/data/ingestion.py	test_split_ratios_sum_to_total	Unit	Calls split() on a 1000-row fixture. Asserts that row counts across train/val/test sum to 1000.	sum(split sizes) == total rows; no data lost.
T05	src/models/base_model	test_forward_pass_output_shape	Unit	Instantiates a concrete model with	output['logits'].shape ==

ID	Module	Test Name	Type	Description	Pass Criteria
	l.py			a minimal test config. Runs forward() on a synthetic batch. Asserts output shape matches expected dimensions.	(batch_size, num_classes).
T06	src/models/base_model.py	test_compute_loss_returns_scalar	Unit	Calls compute_loss() with mock outputs and labels. Validates return is a scalar tensor with requires_grad=True.	loss.ndim == 0; loss.requires_grad is True.
T07	src/models/base_model.py	test_optimizer_step_decreases_loss	Unit	Performs a single forward + backward + optimizer step on synthetic data. Asserts loss decreases (or at least does not increase numerically unstably).	loss_after <= loss_before; no NaN/Inf in loss.
T08	src/training/trainer.py	test_trainer_fit_runs_without_error	Smoke	Runs Trainer.fit() for 2 epochs on synthetic tiny dataset (100	Completes in <30s; returns dict with 'val_loss' key; no exception

ID	Module	Test Name	Type	Description	Pass Criteria
				samples). No GPU required; device=cpu in config.	raised.
T09	src/training/trainer.py	test_checkpoint_save_load_round_trip	Integration	Saves checkpoint after 1 epoch, loads it into a fresh Trainer instance. Asserts model state dicts are identical (torch.allclose).	All parameter tensors match exactly after load; epoch counter restored.
T10	src/evaluation/metrics.py	test_accuracy_metric_correctness	Unit	Calls compute_metrics() with known predictions and references. Validates accuracy against hand-computed expected value.	Computed accuracy matches expected value within 1e-6 tolerance.
T11	src/evaluation/harness.py	test_harness_run_benchmarks_returns_results	Integration	Instantiates Evaluation Harness with a mock model and a single tiny benchmark fixture. Calls run_benchmarks() and asserts result	Returns dict with benchmark name as key; all configured metrics present as float values.

ID	Module	Test Name	Type	Description	Pass Criteria
				structure.	
T12	Full pipeline	test_full_pipeline_smoke	Smoke	Synthesizes raw data → runs DataIngestionPipeline → trains for 1 epoch → runs Evaluation Harness → asserts non-empty report. CPU only, <60 seconds total.	Pipeline completes without error; report.md exists at output_dir; val_loss is a finite float.

8.3 Coverage Requirements

- **Minimum Line Coverage:** 80% of all lines in src/ must be covered by the test suite. This threshold is enforced by the --cov-fail-under=80 flag in ci.yml and will cause the CI job to fail if not met.
- **Coverage Reports:** HTML and XML coverage reports are generated on every CI run. XML reports are uploaded to Codecov, which posts a coverage summary and diff as a comment on every pull request. Coverage trends are tracked over the program lifecycle.
- **Exclusions:** The following paths are excluded from coverage measurement via pyproject.toml [tool.coverage.omit]:
 - notebooks/ — Not importable Python modules.
 - src/*/__init__.py — Package init files with no executable logic.
 - Abstract base class method stubs (decorated with @abstractmethod) — Cannot be executed without a concrete implementation.
 - tests/ themselves — Coverage measures production code, not test code.

- **Branch Coverage:** Branch coverage reporting is enabled (--cov-branch) and tracked but not enforced at a hard threshold. Branch coverage targets are reviewed at the Month 2 program checkpoint.

9. Staffing Plan

The research program is staffed with a compact, high-leverage team. All roles are defined by function rather than individual headcount — a single person may hold multiple roles if appropriately qualified. FTE allocations reflect dedicated effort on this program only; personnel with split responsibilities across programs must have their time explicitly agreed with their line manager before program kickoff.

Role	FTE	Responsibilities	Start Date	End Date	Est. Cost (USD/mo)
Principal Investigator (PI)	0.25	Scientific direction and strategy; hypothesis sign-off; internal and external paper review; publication submission; stakeholder communication; program risk escalation.	Month 1	Month 6	\$4,375 / mo
Research Engineer Lead	1.0	Repository architecture and scaffolding;	Month 1	Month 6	\$16,500 / mo

Role	FTE	Responsibilities	Start Date	End Date	Est. Cost (USD/mo)
		CI/CD pipeline ownership; code review (all PRs); engineering standards enforcement; compute provisioning; dependency management.			
ML Research Scientist	1.0	Model architecture design and implementation; experiment design and hypothesis formulation ; training run execution and analysis; evaluation harness validation; ablation studies; paper authorship.	Month 1	Month 6	\$15,000 / mo
Data Engineer	0.5	Data ingestion pipeline design and implementation; schema and quality validation;	Month 1	Month 4	\$6,500 / mo

Role	FTE	Responsibilities	Start Date	End Date	Est. Cost (USD/mo)
		storage architecture (S3, NFS); data versioning (DVC or equivalent); dataset licensing audit.			
Research Assistant / MLOps	0.5	Experiment tracking workspace administration (W&B / MLflow); hyperparameter sweep configuration and monitoring; compute cost tracking dashboard; notebook execution maintenance; environment reproducibility.	Month 2	Month 6	\$5,000 / mo
Technical Writer	0.25	Model card and dataset card authorship; README and quickstart documentation; notebook narrative prose; API docstring review;	Month 3	Month 6	\$2,500 / mo

Role	FTE	Responsibilities	Start Date	End Date	Est. Cost (USD/mo)
		publication supplemental materials formatting.			

9.1 RACI Matrix

The following RACI matrix defines accountability for key program decisions. **R** = Responsible (does the work); **A** = Accountable (final decision authority); **C** = Consulted (input required); **I** = Informed (notified of outcome).

Decision / Activity	PI	Eng. Lead	ML Scientist	Data Eng.	MLOps / RA	Tech. Writer
Repository Architecture Sign-Off	A	R	C	C	I	I
Experiment Design & Hypothesis Approval	R/A	C	R	I	I	I
Compute Budget Approval (>\$2,500 threshold)	A	R	C	C	C	I
Dataset Selection & Licensing Approval	A	C	C	R	I	I
Model Card & Dataset Card Approval	A	C	C	C	I	R

Decision / Activity	PI	Eng. Lead	ML Scientist	Data Eng.	MLOps / RA	Tech. Writer
Publication Submission Decision	R/A	C	C	I	I	C
CI/CD Quality Gate Configuration Changes	I	R/A	C	I	C	I

10. Risk Register

The following risk register catalogues known program risks as of document version 1.0. Risk Score is computed as Likelihood × Impact using a 3-point scale (H=3, M=2, L=1), yielding scores from 1 (Low×Low) to 9 (High×High). Risks with a score ≥ 6 are flagged for immediate mitigation planning. The register is a living document; owners are required to update status at each monthly program review.

ID	Risk Description	Category	Likelihood	Impact	Score	Mitigation Strategy	Owner	Status
R01	GPU quota shortfall delays training runs; cloud provider	Infrastructure	H	H	9	Submit quota increase request at least 4 weeks before Month	Eng. Lead	Open

ID	Risk Description	Category	Likelihood	Impact	Score	Mitigation Strategy	Owner	Status
	quota limits prevent launching required instance count.					2 training runs. Identify secondary cloud provider as fallback. Pre-validate quota on Day 1.		
R02	Data licensing restrictions limit dataset use in published work or prohibit commercial derivative use.	Legal / Data	M	H	6	Complete license audit for all datasets in Month 1 before data pipeline investment. Identify open-license alternative datasets. Consult legal counsel if attribution-only or	Data Eng.	Open

ID	Risk Description	Category	Likelihood	Impact	Score	Mitigation Strategy	Owner	Status
						NC license is involved.		
R03	Reproducibility failure: notebooks produce different outputs in a new environment due to undeclared dependencies or OS-level variation.	Technical	M	M	4	Enforce conda-lock or pip-compile for fully pinned dependency files. Test notebook execution in a clean Docker image in CI (publish-notebooks.yml). Pin system-level CUDA and cuDNN versions in Docker base image.	Eng. Lead	Open
R04	Key staff attrition: Research	Staffing	L	H	3	Maintain comprehensive	PI	Open

ID	Risk Description	Category	Likelihood	Impact	Score	Mitigation Strategy	Owner	Status
	ch Scientist or Eng. Lead exits the program mid-execution, causing knowledge loss and timeline slip.					documentation and code comments from Day 1. Weekly knowledge-share sessions recorded and archived. Cross-train MLOps/RA on core training loop. Ensure no single person is a unique knowledge owner.		
R05	Experiment tracking SaaS (W&B or MLflow Cloud) experiences outage ,	Infrastructure	L	M	2	Configure local MLflow server on a team instance as offline fallback. Log	MLOps / RA	Open

ID	Risk Description	Category	Likelihood	Impact	Score	Mitigation Strategy	Owner	Status
	blocking run logging and team visibility into experiments.					all hyperparameters and metrics to structured JSON files in the checkpoint directory regardless of tracker status.		
R06	Compute cost overrun: unbounded hyperparameter sweeps or runaway training jobs exceed budget allocation.	Financial	M	H	6	Enforce max_runtime and max_concurrent_runs limits on all sweeps. Configure cloud budget alerts at 50% and 80% thresholds (see Section 3). Review	Eng. Lead	Open

ID	Risk Description	Category	Likelihood	Impact	Score	Mitigation Strategy	Owner	Status
						cost dashboard weekly . Use spot instances for all non-critical runs.		
R07	Benchmark dataset distribution shift: target benchmark dataset changes between program start and submission, invalidating comparative results .	Research	M	H	6	Pin benchmark dataset version at program start using dataset version hashes or DVC. Monitor benchmark maintainer announcements. Run evaluation on both old and new versions if a	ML Scientist	Open

ID	Risk Description	Category	Likelihood	Impact	Score	Mitigation Strategy	Owner	Status
						change occurs. Document dataset version in paper.		
R08	CI pipeline flakiness: intermittent test failures unrelated to code changes erode team trust in CI and lead to bypassed quality gates.	Technical	M	M	4	Tag known flaky tests with @pytest.mark.flaky and quarantine immediately. Investigate and fix root cause within one sprint. Maintain a flaky test count metric; target: zero flaky tests at all times.	Eng. Lead	Open
R09	Security breach of	Security	L	H	3	Use OIDC-based short-	Eng. Lead	Open

ID	Risk Description	Category	Likelihood	Impact	Score	Mitigation Strategy	Owner	Status
	cloud credentials: leaked API keys or AWS access keys result in unauthorized resource usage or data exfiltration.					lived tokens for CI/CD cloud access ; avoid long-lived static credentials. Enforce git-secrets or trufflehog pre-commit hook to detect committed secrets . Rotate all keys every 90 days. Audit IAM permissions quarterly.		
R10	Publication timeline slip: peer review process at	External	H	M	6	Submit to arXiv concurrently with venue submission	PI	Open

ID	Risk Description	Category	Likelihood	Impact	Score	Mitigation Strategy	Owner	Status
	target venue takes longer than anticipated, missing the program's 6-month delivery target.					sion to establish priority. · Identify a backup venue with a later deadline. Begin internal review process no later than Month 5 to ensure submission is ready by Month 6 regardless of external timeline.		

High-Priority Risks Requiring Immediate Action

Risks R01, R02, R06, R07, and R10 each carry a Risk Score of 6 or higher. Mitigation actions for these risks must be initiated in **Week 1** of the program, before any compute or data pipeline work begins. The PI is responsible for

confirming mitigation status at the Month 1 program review.

11. Next-Step Checklist

The following prioritized checklist translates the program plan into concrete, assignable actions organized by time horizon. Items are sequenced to respect upstream dependencies: infrastructure must precede code, code must precede training, training must precede evaluation and publication. Each item should be assigned to a named owner in the team's project tracker (GitHub Issues or equivalent) before program kickoff.

Immediate (Week 1-2)

- Create GitHub organization and repository with branch protection rules (main: require PR, require CI pass, no force-push).
- Set up `pyproject.toml` (project metadata, dependencies, tool config for `pytest`, `ruff`, `mypy`, `black`), `Makefile` with all standard targets, and `.pre-commit-config.yaml`.
- Configure CI workflows: `ci.yml` (lint → unit tests → integration → smoke) and `lint.yml` (pre-commit hooks). Confirm all jobs pass on an empty `src/` skeleton.
- Provision cloud compute instances (CPU for preprocessing, GPU for training). Test SSH access and verify CUDA driver versions. Submit GPU quota increase request if needed.
- Create experiment tracking workspace: W&B project or MLflow server. Add all team members as workspace collaborators. Validate API key secret in GitHub Actions.

- Onboard all team members: GitHub org access, cloud console access, experiment tracker access, Slack/comms channel setup. Distribute this document and confirm all have read Section 3 (budget policy).
- Complete data licensing audit for all candidate datasets. Document findings and obtain PI sign-off before any data is downloaded or processed.
- Configure cloud budget alerts at 50% and 80% thresholds in cloud provider console. Tag all resources with `project=research-program`.

Short-Term (Month 1)

- Implement `DataIngestionPipeline` (`ingestion.py`, `preprocessing.py`, `validation.py`) and run full validation against the licensed target dataset. Fix all validation failures before Month 2.
- Complete `BaseModel` abstract class and implement first concrete architecture (`transformer.py`). Register in `registry.py`. All unit tests T01-T07 must pass.
- Achieve first end-to-end training run with metrics logged to the experiment tracker. Run must be reproducible by a second team member using only the committed config and a single shell command.
- Publish NB01 (data exploration) as an executed notebook via Papermill. Verify notebook is accessible via the published GitHub Pages URL.
- Establish compute budget tracking dashboard in the cloud provider's cost explorer or a shared spreadsheet. Assign MLOps/RA as weekly tracker updater.
- Achieve $\geq 80\%$ unit test coverage on `src/data/` and `src/utils/` modules. Coverage report must pass in CI.

Medium-Term (Month 2-4)

- Complete hyperparameter sweep infrastructure (`configs/sweep/`, `scripts/sweep.sh`). Run first automated sweep on baseline model with at least 20 configurations. Log all results to experiment tracker.

- Build and validate `EvaluationHarness` against at least two published benchmark baselines. Results must match published numbers within expected variance before evaluation is used in paper.
- Achieve $\geq 80\%$ line coverage across all `src/` modules (not just data/utils). Codecov integration must post coverage diff on every PR.
- Execute and publish all five reproducible notebooks (NB01-NB05) to GitHub Pages via `publish-notebooks.yml`. All notebooks must pass execution with zero cell errors.
- Draft model card (`docs/model_card.md`) and dataset card (`docs/dataset_card.md`) in Hugging Face format. PI and Tech. Writer must review and approve before Month 5.
- Integrate `publish-notebooks.yml` workflow fully into CI/CD. Confirm automatic publication triggers on every merge to main.

Longer-Term (Month 5-6)

- Finalize ablation studies (NB04) and hyperparameter sensitivity analysis (NB05). Results must be stable before paper writing begins.
- Complete internal paper draft. Circulate to all team members for review. Incorporate feedback. PI issues final approval for external submission.
- Format manuscript to target venue LaTeX template. Assemble submission package: code repository link, model card, reproducibility checklist, supplemental materials.
- Submit paper to target venue. Simultaneously post to arXiv as an open preprint to establish priority (confirm with PI on embargo policy if applicable).
- Archive final model checkpoints and full dataset to long-term S3 storage with versioned manifests. Update dataset card with final dataset statistics and access instructions.
- Write retrospective document: lessons learned, what worked, what to improve for the next program iteration. Archive alongside final program artifacts.

- Tag final release in GitHub repository (v1.0.0). Create GitHub Release with DOI link (Zenodo) for academic citation of the codebase.

Appendix A: Glossary

Term	Definition
Benchmark	A standardized dataset and evaluation protocol used to compare model performance across research publications. Examples include GLUE, SuperGLUE, ImageNet, and MMLU.
CI/CD	Continuous Integration / Continuous Delivery. A software engineering practice in which code changes are automatically built, tested, and (optionally) deployed on every commit or pull request, using a pipeline defined as code.
Codecov	A SaaS code coverage reporting platform that integrates with GitHub to post coverage summaries and diffs as PR comments. Used to enforce the $\geq 80\%$ coverage threshold in CI.
DVC	Data Version Control. An open-source tool for versioning large datasets and machine learning artifacts alongside source code in Git, using storage backends such as S3 or GCS.
EDA	Exploratory Data Analysis. The initial phase of data analysis in which statistical summaries, distributions, and visualizations are used to understand the structure, quality, and characteristics of a dataset before modeling.
FTE	Full-Time Equivalent. A unit expressing the work capacity of one full-time employee over a given period. 0.5 FTE

Term	Definition
	= half of a full-time employee's available working hours.
Hydra	A Python framework for composable, hierarchical configuration management developed by Meta. Used in this program to manage model, training, and evaluation configurations via structured YAML files with override support.
MLflow	An open-source MLOps platform for experiment tracking, model registry, and artifact logging. Supports local and remote tracking servers. Used as one of two approved experiment trackers in this program.
MLOps	Machine Learning Operations. The set of practices, tools, and culture that enables reliable, reproducible, and scalable delivery of machine learning systems, analogous to DevOps in traditional software engineering.
Model Card	A structured documentation artifact accompanying a trained model. It specifies the model's intended use, training data, evaluation results, limitations, ethical considerations, and license. Standardized by Hugging Face and adopted by NeurIPS.
mypy	A static type checker for Python. When run in strict mode, it enforces that all functions and variables have type annotations and that annotated types are used consistently throughout the codebase.
nbconvert	A Jupyter project tool for converting .ipynb notebook files to other formats including HTML, PDF, and Markdown. Used in the publish-notebooks.yml workflow to render executed notebooks for web publication.
OmegaConf	A Python library for hierarchical configuration management, used as the configuration backend for Hydra. Supports structured configs via dataclasses and YAML-based composition.

Term	Definition
Papermill	A Jupyter project tool for parameterizing and executing Jupyter notebooks programmatically. Enables reproducible notebook execution in CI/CD pipelines by injecting parameter values at runtime without manual cell editing.
PI	Principal Investigator. The senior scientist responsible for the intellectual direction of the research program, including hypothesis formation, experimental design approval, and publication submission decisions.
RACI	A responsibility assignment matrix with four roles: Responsible (does the work), Accountable (final authority), Consulted (provides input), and Informed (notified of outcome). Used to clarify decision ownership.
ruff	An extremely fast Python linter and code formatter written in Rust. Replaces flake8, isort, and partially black in this program's linting stack. Configured via <code>pyproject.toml</code> [<code>tool.ruff</code>].
W&B (Weights & Biases)	A commercial MLOps platform for experiment tracking, hyperparameter sweep management, artifact versioning, and collaborative dashboards. Used as one of two approved experiment trackers. Referred to as "W&B" or "wandb" in code.

Appendix B: Tool and Library Versions

All program dependencies are pinned to the exact versions listed below. These versions are authoritative; deviations require a documented justification and Eng.

Lead approval. The full transitive dependency lock file is maintained separately as `requirements-lock.txt` (generated via `pip-compile`) or `conda-lock.yml`.

Package / Tool	Pinned Version	Role in Program	License
Python	3.11.9	Primary language runtime	PSF
PyTorch	2.3.1	Deep learning framework, model training and inference	BSD-3
CUDA Toolkit	12.4	GPU acceleration for PyTorch on A100 nodes	NVIDIA EULA
Hydra-core	1.3.2	Composable configuration management	MIT
OmegaConf	2.3.0	Configuration backend for Hydra structured configs	BSD-3
MLflow	2.13.2	Experiment tracking, artifact logging, model registry	Apache-2.0
wandb	0.17.3	Weights & Biases SDK for experiment tracking and sweeps	MIT
pandas	2.2.2	Tabular data manipulation in ingestion and validation pipeline	BSD-3
numpy	1.26.4	Numerical operations; used throughout src/	BSD-3
pytest	8.2.2	Test runner for unit, integration, and smoke tests	MIT
pytest-cov	5.0.0	Coverage plugin for pytest; generates XML/HTML reports	MIT
ruff	0.4.10	Fast Python linter	MIT

Package / Tool	Pinned Version	Role in Program	License
		enforcing code quality in CI	
black	24.4.2	Opinionated code formatter; enforced in CI and pre-commit	MIT
isort	5.13.2	Import order formatter; integrated with pre-commit	MIT
mypy	1.10.1	Static type checker enforcing zero-error policy in CI	MIT
nbconvert	7.16.4	Notebook-to-HTML conversion for publication pipeline	BSD-3
papermill	2.6.0	Parameterized notebook execution engine for CI/CD	BSD-3
pre-commit	3.7.1	Git hook manager enforcing lint and format checks locally	MIT
pandera	0.20.1	DataFrame schema validation in DataIngestionPipeline	MIT
pip-tools	7.4.1	Dependency locking via pip-compile to generate requirements-lock.txt	BSD-3

Appendix C: Useful Commands

Reference

The `Makefile` provides standardized shortcuts for all common developer and research workflows. All targets assume the project virtual environment is activated. Run `make help` to print all available targets with descriptions. The table below is the canonical reference.

Make Target	Description	Underlying Command
<code>make install</code>	Install the package and all dependencies (including dev extras) into the active virtual environment.	<code>pip install -e ".[dev]"</code>
<code>make lint</code>	Run all linting and formatting checks: ruff, black (check-only), isort (check-only), and mypy. Reports all violations without modifying files.	<code>pre-commit run --all-files</code>
<code>make format</code>	Auto-format all Python source files in-place using black and isort. Run this before committing to avoid CI lint failures.	<code>black src/ tests/ && isort src/ tests/</code>
<code>make test</code>	Run the full test suite (unit + integration + smoke) with coverage reporting. Fails if coverage drops below 80%.	<code>pytest tests/ --cov=src --cov-report=html --cov-fail-under=80</code>
<code>make test-unit</code>	Run only the unit test suite. Fast (<5 seconds). Use during active development for rapid feedback.	<code>pytest tests/unit/ -v</code>
<code>make test-smoke</code>	Run only the smoke test suite. Validates full pipeline end-to-end on synthetic data. Must complete in under 60	<code>pytest tests/smoke/ -v --timeout=60</code>

Make Target	Description	Underlying Command
	seconds.	
make train	Launch a training run using the default base configuration. Logs metrics to the configured experiment tracker. Accepts config overrides via ARGS.	bash scripts/train.sh
make evaluate	Run the evaluation harness against the latest or specified checkpoint. Generates evaluation report and leaderboard table. Requires CKPT=path/to/checkpoint.pt.	bash scripts/evaluate.sh
make sweep	Launch a Hydra multirun hyperparameter sweep using the configuration in configs/sweep/. Logs all sweep runs to experiment tracker under a named sweep group.	bash scripts/sweep.sh
make notebook	Execute all notebooks in order using Papermill and place executed outputs in notebooks/executed/. Requires all notebook dependencies to be installed.	papermill notebooks/*.ipynb ...
make notebook-convert	Convert all executed notebooks in notebooks/executed/ to HTML using nbconvert and place outputs in docs/notebooks/.	jupyter nbconvert --to html ...
make ingest	Run the full DataIngestionPipeline: load raw data, validate, split, and save processed artifacts to data/processed/.	python -m src.data.ingestion
make clean	Remove all generated artifacts: Python __pycache__, .pytest_cache, htmlcov/, dist/,	find . -name __pycache__ -exec rm -rf {} + ...

Make Target	Description	Underlying Command
	.mypy_cache/, and notebooks/executed/. Does not delete data/ or checkpoints/.	
make clean-data	Remove all processed data artifacts in data/processed/ only. Does not touch data/raw/. Use to force a full re-ingestion run.	rm -rf data/processed/*
make docs	Build the Sphinx or MkDocs documentation site from docs/ source files and API docstrings. Output placed in docs/_build/ or site/.	mkdocs build or sphinx-build
make help	Print all available Makefile targets with descriptions. Generated automatically from Makefile comment annotations.	grep -E '^[a-zA-Z_-]+:.*?##' Makefile